

Nicolas Tito Dallmann
Pedro Henrique Silva do Amaral
Sione Serrano Pedro
Thiago Jaunatre Faquinello

O objetivo principal era fazer um programa em linguagem C capaz de ler um arquivo de texto ASCII, contar a frequência de cada palavra e, em seguida, permitir que o usuário realizasse consultas e listagens diversas. O desafio era garantir que o desempenho fosse o melhor possível, respeitando regras de complexidade computacional, como $O(n \log n)$ para leitura do arquivo, o que inclui a leitura, armazenamento e ordenação de dados, $O(1)$ para consultas diretas e $O(n)$ listagem das palavras do arquivo, de acordo com determinado critério.

Para fazer isso, tendo em vista a complexidade $O(1)$ para consultas, utilizamos uma abordagem baseada no armazenamento dos dados em uma tabela hash com tratamento de colisões por sondagem linear, que garante buscas rápidas. Como também era necessário realizar listagens em determinadas ordens, nós criamos vetores contendo os mesmos dados, que, depois, seriam organizados pelo algoritmo de ordenação Mergesort, que nos permite ordená-los com a complexidade $O(n \log n)$ de forma garantida.

A montagem do programa começou com uma leitura inicial do arquivo, na qual é realizada a contagem do número de palavras presentes, afinal, antes de criar a tabela hash, o programa precisa saber quantas palavras existem no arquivo para dimensionar corretamente a tabela. Dessa forma, a tabela hash do programa consegue sempre se “adaptar” ao tamanho do arquivo lido. Essa leitura inicial é simples e rápida: abrimos o arquivo, inicializamos um contador em zero, e enquanto conseguimos ler uma palavra, o contador é incrementado. Ao final, o arquivo é fechado e o número total de palavras é retornado.

Pseudocódigo: Contar Palavras

abrir arquivo

contador = 0

enquanto conseguir ler uma palavra:

 contador++

fechar arquivo

retornar contador

Com o número total de palavras em mãos, criamos a tabela hash com um tamanho igual ao dobro do número de palavras do arquivo, de modo a reduzir o número de colisões. Ela é inicialmente preenchida com NULL em todas as posições, indicando que nenhuma palavra foi armazenada ali ainda. Em seguida, lemos o arquivo de novo, agora de forma definitiva, e cada palavra é inserida na tabela hash. Para isso, geramos uma chave numérica somando os caracteres da palavra e aplicamos o módulo pelo tamanho da tabela para obter a posição ideal.

O processo de inserção é crucial para o desempenho: como cada elemento da tabela foi definido como uma struct, que contém a palavra e seu número de ocorrências, se a posição estiver vazia, a palavra é inserida com o contador de ocorrências em 1. Além disso, nesse caso, é incrementado 1 também à variável “qtd”, que conta o número de palavras distintas. Se a palavra já estiver lá, simplesmente somamos uma unidade ao contador. Se houver uma colisão com outra palavra (ou seja, palavras diferentes caem na mesma posição), usamos o sondagem linear para procurar a próxima posição disponível. Ao final desse processo, todas as palavras distintas ficam armazenadas, cada uma com sua contagem final.

Pseudocódigo da Inserção

para cada palavra do arquivo:

 pos = hash(palavra)

 enquanto posição ocupada E palavra diferente:

 pos = próxima posição (circular)

 se posição vazia:

 armazenar palavra com ocorrencias = 1

 senão:

 ocorrencias++

Para atender aos requisitos de listagem rápida, criamos dois vetores. Copiamos todas as posições preenchidas para eles, resultando em dois vetores idênticos, com tamanho igual ao número de palavras distintas no arquivo, número esse fornecido pela variável “qtd” da tabela hash. Em seguida, usamos o algoritmo Mergesort para ordená-los: um vetor é ordenado por número de ocorrências, e o outro é ordenado pela ordem alfabética. A necessidade de ordenar de acordo com critérios distintos fez com que fosse necessário a criação de dois algoritmos Mergesort distintos.

Assim, é finalizada a etapa de leitura do programa. Ao analisar a ordem de complexidade média de cada função usada nessa parte, chega-se ao seguinte:

“contar_palavras” - $O(n)$; “cria_hash” - $O(n)$; “gera_hash” - $O(1)$; “inserir_na_tabela” - $O(n)$; “cria_vetor” - $O(n)$; cada Mergesort - $O(n \log n)$. A complexidade que “comanda” o funcionamento da etapa de leitura é a maior, ou seja, $O(n \log n)$, mostrando que o programa respeita o limite do enunciado.

O programa entra então em um *loop* contínuo, exibindo o menu até que a opção 6 seja selecionada.

A Consulta Direta (Opção 1), quando o usuário digita uma palavra, é feita utilizando a tabela hash. Calculamos a posição hash e percorremos as posições subsequentes apenas em caso de colisões. Esse processo tem complexidade média $O(1)$, atendendo ao requisito de consulta rápida.

Pseudocódigo da Consulta

pos = hash(palavra)

enquanto verificou menos que TABLE_SIZE:

 se posição contém palavra procurada:

 retornar ocorrencias

 ir para próxima posição

retornar -1 (não encontrada)

As outras opções utilizam os vetores que já estão ordenados. A listagem crescente de ocorrências (Opção 2) e a decrescente (Opção 3) são feitas percorrendo o vetor ordenado por ocorrências do início ao fim e do fim para o início, respectivamente. A listagem alfabética (Opção 4) percorre o vetor ordenado alfabeticamente. A listagem por letra inicial (Opção 5) simplesmente filtra o vetor de ocorrências, comparando apenas o primeiro caractere. Todas essas impressões se baseiam em simplesmente percorrer os vetores formados e, portanto, têm complexidade $O(n)$, satisfazendo os requisitos.

Por fim, foi utilizada a função “free” para que a memória utilizada para a execução do programa fosse liberada, após o comando de encerramento.

Em resumo, a implementação respeita os limites, porque a leitura do arquivo com a ordenação inicial está dentro do limite de $O(n \log n)$ (dominado pelo Mergesort). A consulta de ocorrência atinge $O(1)$ graças à tabela hash. Por fim, as listagens já ordenadas são apenas percursos simples, resultando em $O(n)$. Assumimos, conforme as orientações, que o arquivo não contém caracteres acentuados e nem sinais de pontuação

No entanto, é importante deixar claro que existe um pior caso, no qual todas as palavras são diferentes, mas possuem um hash idêntico, causando um número excessivo de colisões. Como resultado, a função “inserir_na_tabela” acaba tendo uma complexidade maior, pelo fato de, enquanto percorre o arquivo ($O(n)$), precisar percorrer toda a tabela hash para encontrar a posição adequada para determinada palavra, através de sondagem linear ($O(n)$), resultando em uma complexidade de ordem $O(n^2)$. Algo semelhante ocorre com a consulta de ocorrências de determinada palavra, pois a função “consulta_ocorrencias”, que deveria ter complexidade $O(1)$, poderá ter que percorrer a tabela inteira para encontrar determinada palavra, resultando em uma ordem de complexidade $O(n)$. O programa apresenta esse desempenho abaixo do esperado nesse caso, pois o funcionamento perfeito de uma tabela hash depende da inexistência de colisões, o que é impossível garantir.

Por fim, é possível concluir que o programa desenvolvido cumpre as exigências do enunciado, organizando dados de maneira eficiente, com o uso de estruturas de dados e algoritmos de ordenação.